1 What is Spring Boot?

- Spring Boot is a framework built on top of the Spring Framework that simplifies Java application development.
- It reduces the need for complex configuration and setup.
- It helps developers build production-ready applications quickly with minimal effort.

2 Why is Spring Boot used?

- Spring Boot is used to speed up application development by reducing boilerplate code and manual configuration.
- It provides built-in tools and configurations that make development easier and more efficient.
- It is widely used for building web applications, REST APIs, microservices, and enterprise applications.

3 What is the difference between Spring and Spring Boot?

- Spring requires more manual configuration and setup using XML files or Java configuration.
- Spring Boot provides automatic configuration based on the project dependencies and sensible default settings.
- Spring Boot makes development faster by reducing setup effort, boilerplate code, and configuration.

4 What are the main features of Spring Boot?

- Spring Boot provides auto-configuration to reduce manual setup work.
- It includes starter dependencies that simplify dependency management.
- It offers embedded servers so applications can run without external deployment.
- It provides production-ready features like monitoring, logging, health checks, and metrics.

5 What is auto-configuration in Spring Boot?

- Auto-configuration automatically sets up application components based on the dependencies present in the project.
- It reduces the amount of configuration developers need to write manually.
- It helps applications start faster with sensible default settings.

6 What are starter dependencies in Spring Boot?

- Starter dependencies are pre-configured dependency packages provided by Spring Boot.
- They help developers add required libraries with a single dependency.
- This reduces dependency conflicts and simplifies project setup.

7 What is an embedded server in Spring Boot?

- An embedded server is a built-in web server included inside the application.
- Spring Boot commonly uses Tomcat, Jetty, or Undertow as embedded servers.
- This removes the need to deploy applications separately on external servers.

8 What is application.properties in Spring Boot?

- application.properties is a configuration file used to define application settings.
- It stores values like database connection details, server port, and logging configuration.
- It helps manage application settings in one central place.

9 What is Spring Initializr?

- Spring Initializr is a web tool provided by Spring to generate Spring Boot projects.
- It helps to create a project skeleton with required dependencies and project structure.
- It saves time and simplifies the initial setup of a Spring Boot application.

10 What is the purpose of @SpringBootApplication?

- @SpringBootApplication is a combination of @Configuration, @EnableAutoConfiguration, and @ComponentScan annotations.
- It is used on the main class to enable Spring Boot auto-configuration.
- It also scans the base package for components and configurations.

11 What is dependency injection in Spring Boot?

- Dependency Injection (DI) is a design pattern where Spring provides the objects that a class needs.
- Spring Boot uses DI to manage and inject dependencies automatically.
- It reduces coupling and makes the application more modular and testable.

12 What is the purpose of @RestController?

- @RestController is a stereotype annotation used to create RESTful web services.
- It is a combination of @Controller and @ResponseBody.
- It indicates that the class handles HTTP requests and returns data in the response body.

13 What is the purpose of @Service annotation?

- @Service is a stereotype annotation used to mark a class as a service layer component.
- It indicates that the class contains business logic.
- It is a specialization of @Component.

14 What is the purpose of @Repository annotation?

- @Repository is used to mark a class as a data access layer component.
- It indicates that the class is responsible for database operations.
- It also enables exception translation for database-related exceptions.

15 What is the purpose of @Configuration annotation?

- @Configuration is used to indicate that a class contains Spring configuration.
- The class can define one or more @Bean methods to create and manage beans.
- It is an alternative to XML-based configuration.

16 What is the purpose of @Bean annotation?

- @Bean is used in a @Configuration class to define a bean.
- It tells Spring to create an object of the method's return type and manage it in the application context.
- It is useful when third-party classes cannot be annotated with Spring annotations.

17 What is the purpose of @Autowired annotation?

- @Autowired is used to automatically inject dependencies into a bean.
- It can be used on constructors, fields, or setter methods.
- It tells Spring to resolve and inject the required bean from the application context.

18 What is the purpose of @Value annotation?

- @Value is used to inject values from properties files or environment properties.
- It can be used on fields, constructor parameters, or setter methods.
- It helps externalize configuration by mapping property values to variables.

19 What is the purpose of @RequestMapping annotation?

- @RequestMapping is used to map HTTP requests to handler methods in a controller.
- It can be applied at the class level or method level.
- It supports various HTTP methods like GET, POST, PUT, DELETE, etc.

20 What is the purpose of @PathVariable annotation?

- @PathVariable is used to extract values from the URI path.
- It is typically used in RESTful web services.
- It helps map dynamic URL segments to method parameters.

21 What is the purpose of @RequestParam annotation?

- @RequestParam is used to extract query parameters from the request.
- It binds the value of a request parameter to a method parameter.
- It can also be used to set default values for optional parameters.

22 What is the purpose of @RequestBody annotation?

- @RequestBody is used to bind the HTTP request body to a method parameter.
- It is commonly used in POST, PUT, and PATCH requests.
- It automatically converts the request body (usually JSON) into a Java object.

23 What is the purpose of @ResponseBody annotation?

- @ResponseBody is used to return the response directly in the body.
- It converts the returned object into JSON or XML and sends it to the client.
- It is often used in combination with @Controller.

24 What is the purpose of @CrossOrigin annotation?

- @CrossOrigin is used to enable Cross-Origin Resource Sharing (CORS).
- It allows requests from different domains to access the application's resources.
- It can be applied at the class level or method level.

25 What is the purpose of @ExceptionHandler annotation?

- @ExceptionHandler is used to handle exceptions in a centralized way.
- It can be used in a controller or in a @ControllerAdvice class.
- It maps specific exceptions to custom response messages or status codes.

26 What is the purpose of @ControllerAdvice annotation?

- @ControllerAdvice is used to define global exception handling and other cross-cutting concerns.
- It can be used to handle exceptions, bind data, or modify responses globally.
- It helps reduce code duplication across multiple controllers.

27 What is the purpose of @Profile annotation?

- @Profile is used to define environment-specific beans.
- It allows you to register beans only for certain profiles (e.g., dev, test, prod).
- It helps manage different configurations for different environments.

28 What is the purpose of @Conditional annotation?

- @Conditional is used to register a bean only if a certain condition is met.
- It is part of Spring's conditional configuration.
- It helps create flexible and dynamic configurations based on custom logic.

29 What is the purpose of @Transactional annotation?

- @Transactional is used to manage database transactions.
- It ensures that a series of operations either succeed or fail as a whole.
- It can be applied at the service layer or method level.

30 What is the purpose of @EnableAutoConfiguration annotation?

- @EnableAutoConfiguration is used to enable Spring Boot's auto-configuration.
- It tells Spring Boot to automatically configure the application based on the dependencies in the classpath.
- It is usually added implicitly by @SpringBootApplication.

31 What is the purpose of @ComponentScan annotation?

- @ComponentScan is used to specify the base packages to scan for components.
- It helps Spring find and register @Component, @Service, @Repository, @Controller, etc.
- It is optional with @SpringBootApplication as it already includes default component scanning.

32 What is the purpose of @Entity annotation?

- @Entity is used to mark a class as a JPA entity.
- It indicates that the class is mapped to a database table.
- It is used in the persistence layer with Spring Data JPA or Hibernate.

33 What is the purpose of @Table annotation?

- @Table is used to specify the table name and other table-related properties for an entity.
- It can be used when the entity name and table name are different.
- It can also define schema, catalog, and unique constraints for the table.

34 What is the purpose of @Id annotation?

- @Id is used to mark a field as the primary key of an entity.
- It uniquely identifies each record in the database table.
- It is used in combination with other annotations like @GeneratedValue.

35 What is the purpose of @GeneratedValue annotation?

- @GeneratedValue is used to specify the strategy for primary key generation.
- It works with @Id to automatically generate the value for the primary key.
- Common strategies include IDENTITY, SEQUENCE, TABLE, and AUTO.

36 What is the purpose of @Column annotation?

- @Column is used to map a field to a column in the database table.
- It can be used to customize column properties like name, length, nullable, unique, etc.
- It is optional if the field name and column name are the same.

37 What is the purpose of @Repository annotation?

- @Repository is used to mark a class as a repository (data access layer).
- It indicates that the class is responsible for interacting with the database.
- It also enables Spring to translate database exceptions into Spring's DataAccessException hierarchy.

38 What is the purpose of @Query annotation?

- @Query is used to define custom SQL or JPQL queries in repository methods.
- It allows writing custom queries instead of using derived query methods.
- It can be used with named parameters for dynamic values.

39 What is the purpose of @Modifying annotation?

- @Modifying is used with @Query for update, delete, or insert operations.
- By default, @Query is for select queries; this annotation enables modifying queries.
- It should be used with @Transactional to ensure the changes are committed.

40 What is the purpose of @Param annotation?

- @Param is used to bind method parameters to named parameters in a query.
- It is used with @Query for named parameter binding.
- It helps improve readability and maintainability of custom queries.